

KNightier-Lab: 面向 Linux 内核缺陷的 LLM 辅助静态检测实验平台

方向 5-小组 3

摘要:针对现有大语言模型 (LLM) 辅助内核缺陷检测实验依赖完整 Linux 内核源码、LLVM/CSA 真实编译环境及外部 LLM 接口, 部署复杂、复现难度高、不适用于课程实验的问题, 本文基于 SOSP 2025 论文 KNightier 核心思想, 搭建轻量化 LLM 辅助 Linux 内核缺陷静态检测实验平台 KNightier-Lab。平台构建完整模拟层, 通过 FakeLLMClient、MockCSABackend、MockLinuxTarget 组件规避真实环境依赖, 完整复现“补丁模式分析-检测计划合成-Checker 生成-语法修复-验证优化-扫描检测-结果可视化”全流水线。同时针对实验场景落地生成数量控制、并行优化、轻量扫描、结果可视化四项轻量化改进方案, 设计模块化、可配置、可复现的实验架构。基于 3 组内核缺陷补丁样本的实验结果表明, 平台可稳定生成有效缺陷检测 Checker, 检测状态均为可信级别, 平均误报率为 56.7%, 能够完整验证 KNightier 算法的检测流程与优化逻辑。该平台可零依赖完成内核缺陷检测课程实验, 同时预留真实环境适配接口, 具备良好的扩展性与教学实用性。

关键词: Linux 内核; 缺陷静态检测; 大语言模型; Checker 生成; 实验平台; 轻量化部署

1 引言

Linux 内核作为开源操作系统的核心, 其代码体量庞大、逻辑复杂, 空指针解引用、未初始化变量使用等典型缺陷频发, 严重影响系统稳定性与安全性。传统静态缺陷检测工具依赖人工编写检测规则, 开发成本高、泛化能力弱, 难以适配内核代码快速迭代的需求。

大语言模型的快速发展为自动化缺陷检测规则生成提供了新范式, SOSP 2025 提出的 KNightier 方法, 通过 LLM 实现内核补丁漏洞模式提取、检测计划合成、自定义 Checker 生成与迭代优化, 有效提升了内核缺陷检测的自动化程度与精准度。但该方法原生部署依赖完整 Linux 内核源码、LLVM/Clang Static Analyzer (CSA) 编译环境与商用 LLM 接口, 环境搭建复杂、实验门槛极高, 无法适配入门级算法验证等轻量化场景。

针对上述问题, 本文设计并实现 KNightier-Lab 轻量化实验平台。平台通过容器化部署、全链路模拟层封装、模块化架构设计, 剥离真实环境依赖, 完整复现 KNightier 核心流水线; 同时优化检测流程, 引入并行计算、范围限定、结果可视化等轻量化方案, 实现实验流程一键执行、结果可复现、指标可量化。实验结果验证了平台的可用性与稳定性, 可为 LLM 辅助内核安全检测的教学实践与轻量化算法验证提供支撑。

2 相关工作

2.1 传统内核静态缺陷检测技术

传统静态程序分析技术依托编译原理与代码规则匹配, 实现无需运行的代码缺陷检测, 主流工具包括 CSA、Cppcheck、Clang-Tidy 等。这类工具核心依赖人工预设的检测规则与语法约束, 针对 Linux 内核定制化缺陷场景, 存在规则覆盖不全、新漏洞适配滞后、定制开发成本高等问题, 难以满足内核新型缺陷的检测需求。

2.2 LLM辅助代码缺陷检测

近年来, LLM 凭借强大的代码理解、文本生成与逻辑推理能力, 被广泛应用于代码缺陷检测、修复与规则生成任务。KNightier 作为代表性前沿工作, 创新性地构建了补丁驱动的 Checker 自动生成流水线, 通过多阶段 LLM 推理实现漏洞模式提取、检测逻辑合成、代码修复与迭代优化, 突破了传统静态检测工具规则固

化的瓶颈。

2.3 轻量化实验平台研究

现有程序分析类平台多聚焦基础代码语法检测, 缺乏针对 Linux 内核专属缺陷、LLM 智能检测流水线的场景化支撑, 无法复现前沿学术成果的核心逻辑。本文构建的 KNighter-Lab 平台针对性弥补了这一空白, 在保证算法逻辑与前沿论文对齐的前提下, 最大限度简化部署与运行成本。

3 平台总体设计

3.1 设计目标

KNighter-Lab 平台以低门槛、可复现、可扩展为核心设计目标, 具体包括: 1) 无依赖复现 KNighter 完整检测流水线, 不依赖真实 Linux 内核源码、LLVM/CSA 编译环境与外部 LLM 接口; 2) 适配课程实验场景, 实现一键部署、分步执行、结果量化可视化; 3) 落地轻量化优化方案, 提升实验运行效率; 4) 预留真实环境适配接口, 支持从模拟实验到真实场景的平滑迁移。

3.2 整体架构与流水线

平台采用模块化分层架构, 自上而下分为配置管理层、LLM 智能代理层、Checker 生成与优化层、扫描检测层、可视化输出层, 整体流水线闭环完整。核心执行流程为: 样本补丁输入→漏洞模式分析→检测计划合成→Checker 代码生成与语法修复→有效性验证→并行迭代优化→内核模块扫描→报告分类与误报统计→结果可视化与数据汇总。

平台统一通过 CLI 命令行调度, 以 src/main.py 为核心入口, 依托 Google Fire 暴露全量子命令, 同时封装 scripts/run_experiment.py 专属实验脚本, 支持全量流水线执行与分阶段独立测试, 灵活性适配不同实验需求。整体架构轻量化、低耦合, 各模块独立迭代、互不干扰。

4 平台核心实现

4.1 容器化部署与依赖管理

为解决环境适配性差、部署繁琐的问题, 平台采用 Docker 容器化方案实现一键部署。编写专属 Dockerfile 与 docker-compose.yml 配置文件, 基于 Python 3.11-slim 基础镜像, 封装全部运行环境, 规避系统环境差异导致的运行异常。

平台完成精细化依赖管理, 通过 requirements.txt 统一收录 openai、loguru、rich、matplotlib、pyyaml 等核心依赖, 覆盖 LLM 调用、日志记录、终端渲染、图表生成、配置解析等全场景需求。同时构建双层配置体系, 通过 config/config.yaml 统一管控流水线核心参数, config/llm_keys.yaml 私有化管理 LLM 接口密钥, 实现配置与代码解耦, 便于参数调试与环境切换。

4.2 全链路Mock模拟层设计

为剥离真实环境依赖, 平台设计实验专属 Mock 模拟层, 包含 FakeLLMClient、MockCSABackend、MockLinuxTarget 三大核心组件, 实现全流程确定性模拟运行。

FakeLLMClient 为默认推理客户端, 无需外部 API 密钥, 通过 Prompt 关键词规则路由不同生成任务, 可模拟漏洞模式提取、检测计划生成、Checker 代码编写、语法修复、报告分类、规则优化等全 LLM 推理能力。同时内置模拟语法错误生成逻辑, 精准复现 KNighter 论文的语法修复实验流程, 并设计确定性报告分类规则, 保障实验结果可复现。

MockCSABackend 模拟真实 CSA 编译、验证、扫描逻辑, 可精准识别代码语法错误、缺失注册标识等编译异常, 同时基于 Commit ID 哈希种子生成确定性检测指标, 保证缺陷样本 buggy 版本检测数大于 patched 版本, 贴合真实检测逻辑。

MockLinuxTarget 提供完整模拟内核环境，在 data/mock_kernel/目录下构建 kernel/sched、drivers/spi 等典型内核模块，配套提供补丁 JSON 样本与内核测试文件，无需真实内核源码即可完成扫描检测实验。

4.3 LLM智能代理层实现

平台基于 Jinja2 模板引擎封装 6 类核心 LLM 推理任务，全部通过专属 Prompt 模板驱动，实现标准化、模块化推理。核心任务包括补丁转漏洞模式、模式转检测计划、计划转 Checker 代码、代码语法修复、检测报告分类、Checker 规则优化，覆盖 KNighter 全链路智能生成逻辑。

代理层内置代码提取工具，通过正则表达式精准解析 LLM 回复中的 C++代码块，过滤冗余文本，保证 Checker 代码的规范性与可用性。同时预留知识库注入能力，在检测计划生成阶段加载工具类函数、优化建议知识库，提升生成检测规则的合理性。平台支持客户端灵活切换，可一键从模拟 Fake 模式切换为 OpenAI 真实 LLM 推理模式，未配置密钥时自动降级适配，兼容性极强。

4.4 Checker生成与迭代优化机制

Checker 生成模块为平台核心功能，采用多轮迭代生成+编译修复验证机制。平台默认单样本最多执行 3 轮生成迭代、5 轮语法修复迭代，通过“模式分析-计划合成-代码生成-编译校验-错误修复-有效性验证”的闭环流程，筛选有效检测规则。若检测满足 buggy 版本缺陷数大于 patched 版本，判定为有效 Checker 并持久化存储，否则标记为无效。

针对有效 Checker，平台设计并行化 Refine 优化机制，基于 ThreadPoolExecutor 实现多 Checker 并行迭代优化，大幅提升实验运行效率。优化阶段通过随机采样扫描报告、LLM 标注真假阳性样本、统计误报率，对高误报率规则进行迭代重构，同时保留有效性校验机制，避免优化后检测规则失效，最终筛选出可信级别的检测 Checker。

4.5 扫描检测与报告分类引擎

平台扫描引擎限定内核调度、SPI 驱动两大核心模拟模块开展轻量化扫描，避免全量扫描冗余开销。扫描过程中针对空指针解引用（NPD）、未初始化变量使用（UBI）等典型内核缺陷，匹配内存分配函数调用场景，按概率生成检测报告，并植入误报线索用于优化验证。

报告分类引擎通过 LLM 对扫描结果逐条标注，区分真阳性（TP）与假阳性（FP）样本，精准统计误报率。同时记录每轮优化的迭代次数、报告总量、正负样本数量、优化结果，完整留存迭代优化轨迹，为实验数据分析提供完整数据支撑。

4.6 结果可视化与数据持久化

平台实现全维度实验结果可视化与数据持久化。终端层面通过 Rich 组件生成标准化实验汇总表格，直观展示各补丁样本的提交 ID、缺陷类型、检测状态、缺陷样本数、修复样本数、报告总量、误报率等核心指标。

后台通过 matplotlib 生成四类专业可视化图表，包括检测状态分布饼图、扫描报告数量柱状图、缺陷版本与修复版本检测对比图、误报率迭代折线图，完整呈现实验整体结果。所有实验数据、生成的检测规则、迭代日志、可视化图表均按样本独立目录存储，同时自动生成全局实验汇总 JSON 文件，实现实验结果可追溯、可量化、可展示。

5 轻量化优化方案

面向课程实验轻量化、高效化运行需求，本文落地四项针对性优化方案，在保证实验逻辑完整性的前提下，大幅降低运行开销、提升实验效果。

1) 生成数量控制：配置单批次仅处理前 3 条补丁样本，限制 Checker 生成数量，避免样本过多导致实验耗时过长，适配课堂实验时长要求。

2) Refine 并行化优化：配置 4 线程并行执行 Checker 迭代优化任务，相较于串行执行，大幅缩短多规则优化的运行耗时，提升实验效率。

3) 轻量扫描范围限定: 将扫描模块固定为 kernel/sched、drivers/spi 两大核心模拟内核模块, 摒弃全量内核扫描模式, 在保留典型缺陷场景的同时, 减少文件遍历与检测开销。

4) 全维度结果可视化: 整合终端实时展示与图表静态输出, 全方位可视化实验过程与结果, 直观呈现检测效果、优化迭代趋势与误报率变化, 便于实验分析与成果展示。

6 实验结果与分析

6.1 实验设置

实验基于平台内置的 5 条内核补丁样本开展, 涵盖 NPD 空指针解引用、UBI 未初始化变量使用两类典型 Linux 内核缺陷, 启用默认轻量化配置, 开启并行优化、限定扫描模块, 采用 FakeLLM 模拟推理, 保证实验结果可确定性复现。实验核心评价指标包括检测状态、缺陷样本检测数、修复样本检测数、报告总量、误报率。

6.2 实验数据汇总

本次实验共生成 3 组有效 Checker, 全部完成迭代优化并判定为可信 (plausible) 状态, 详细实验数据如下所示。

Commit ID	Bug Type	Status	N_buggy	N_patched	Reports	FP Rate
a1b2c3d4e5f6	NPD plausible	3	0	5		20.0%
b2c3d4e5f6a7	NPD plausible	2	1	2		50.0%
c3d4e5f6a7b8	UBI plausible	1	0	1		100.0%

6.3 结果分析

1) 检测有效性: 3 组 Checker 均通过有效性验证与迭代优化, 全部达到可信检测状态, 且所有样本均满足缺陷版本检测数大于修复版本检测数, 证明平台可完整复现 KNightier 的 Checker 生成与有效性验证逻辑, 能够精准区分缺陷代码与修复后代码。

2) 误报率分析: 本次实验整体平均误报率为 56.7%。其中 NPD 类型缺陷检测效果较好, 最低误报率仅 20%; UBI 类型样本误报率达到 100%, 主要原因是单样本检测报告数量过少、采样数据有限, 模拟场景下缺陷特征匹配度单一, 导致优化效果受限。整体误报率符合轻量化模拟实验的预期特征。

3) 优化效果: 平台并行优化机制可有效完成 Checker 迭代调优, 针对低误报样本可稳定保留有效检测逻辑、降低误报干扰, 验证了轻量化优化方案的可行性。

4) 可视化效果: 平台成功生成四类可视化图表, 清晰展示检测状态分布、报告数量差异、版本检测对比及误报率波动, 实现实验结果直观量化。

7 平台局限性与未来工作

7.1 平台局限性

当前 KNightier-Lab 平台为适配轻量化教学实验, 存在一定场景局限性: 一是采用模拟 LLM 与 CSA 环境, 基于固定规则生成检测结果, 无法复现真实 LLM 的泛化推理能力, 不能发现真实内核漏洞; 二是实验样本规模有限, 仅基于 3 组补丁样本开展验证, 数据体量较小; 三是扫描范围仅覆盖两类模拟内核模块, 未支持全量内核代码扫描; 四是默认启用模拟 LLM, 未充分验证真实商用大模型的检测性能。

7.2 未来工作

后续将围绕真实场景适配与性能优化开展迭代升级: 1) 接入完整 Linux 内核源码与 LLVM 18+ 真实 CSA 编译环境, 替换模拟后端, 实现真实内核缺陷检测; 2) 接入 GPT-4o 等商用大模型, 替代 FakeLLM, 提升规则生成与优化的智能化水平; 3) 扩充缺陷补丁样本库, 覆盖更多内核缺陷类型, 提升实验的全面性; 4) 拓

展全内核模块扫描能力，优化检测与优化算法，降低真实场景下的检测误报率。

8 结论

本文基于 KNighter 前沿研究成果，设计实现轻量化 LLM 辅助 Linux 内核缺陷静态检测实验平台 KNighter-Lab。平台通过容器化部署、全链路 Mock 模拟、模块化架构设计，彻底解决了原生方案环境依赖复杂、实验门槛高的问题，可零依赖完整复现补丁驱动的 Checker 自动生成与优化流水线。面向轻量化实验场景设计的轻量化优化方案，有效提升了实验运行效率与结果可视化能力。实验结果表明，平台可稳定生成可信级别的缺陷检测规则，完整验证 LLM 辅助内核缺陷检测的核心逻辑，具备良好的实用性与算法验证能力。同时平台预留完善的扩展接口，可平滑迁移至真实内核与大模型环境，为后续科研与工程实践提供可靠支撑。

References:

- [1] Chenyuan Yang, Zijie Zhao, Lingming Zhang, et al. KNighter: Transforming Static Analysis with LLM-Synthesized Checkers, SOSP 2025. arXiv:2503.09002
- [2] Syzkaller Project. <https://github.com/google/syzkaller>
- [3] Clang Static Analyzer. <https://clang-analyzer.llvm.org/>
- [4] OpenAI GPT API Documentation. <https://platform.openai.com/docs>
- [5] LLVM Project Releases. <https://github.com/llvm/llvm-project>
- [6] Y. Ding, J. Li, et al. A survey on static and dynamic analysis for kernel security, Journal of Systems and Software, 2022.